

**Universidad Siglo 21**

**Seminario de Práctica de Informática (INF275 - 46312)**

**Seguimiento de vulnerabilidades por proyecto**

**Trabajo práctico 2**

Análisis, diseño, implementación, pruebas y base de datos

Daniel Sebastián Rodríguez

Legajo VINFO16869 | DNI 43.731.653

Licenciatura en Informática | Modalidad EDH

Profesor Pablo Alejandro Virgolini

28 de septiembre de 2026

## Índice

|                                                      |    |
|------------------------------------------------------|----|
| 1. Título, introducción y justificación . . . . .    | 3  |
| 2. Definiciones del proyecto y del sistema . . . . . | 3  |
| 3. Elicitación . . . . .                             | 4  |
| 4. Conocimiento del negocio . . . . .                | 5  |
| 5. Propuesta de solución . . . . .                   | 7  |
| 6. Requerimientos . . . . .                          | 8  |
| 7. Casos de uso . . . . .                            | 10 |
| 8. Etapa de análisis . . . . .                       | 12 |
| 9. Etapa de diseño . . . . .                         | 13 |
| 10. Etapa de implementación . . . . .                | 13 |
| 11. Etapa de pruebas . . . . .                       | 14 |
| 12. Prototipos de interfaz . . . . .                 | 17 |
| 13. Base de datos . . . . .                          | 18 |
| 14. Definiciones de comunicación . . . . .           | 21 |
| 15. Trazabilidad y referencias . . . . .             | 21 |
| Anexo A. Preguntas de elicitación . . . . .          | 23 |

## **1. Título, introducción y justificación**

El proyecto se llama «Sistema de seguimiento de vulnerabilidades por proyecto». Propongo desarrollar una aplicación de escritorio en Java para registrar las vulnerabilidades que se detectan en los recursos informáticos, calcular su severidad con CVSS 3.1 y seguir su corrección hasta el cierre. Los hallazgos se organizan por proyecto y por activo (el administrador gestiona los registros y las cuentas, y cada cliente consulta solamente las vulnerabilidades de sus proyectos).

El contexto es el área tecnológica de Derechos Del Fútbol, donde hago tareas de administración de sistemas, bases de datos y herramientas internas. Elegí este tema porque tiene relación directa con mi trabajo y porque junta en una misma aplicación registro de datos, cálculo, permisos por rol y seguimiento de tareas.

### **1.1 Problema y justificación**

Hoy los hallazgos de seguridad del área se anotan en una planilla y se comunican por correo o mensajería. Con pocos casos ese método alcanza, pero cuando hay varios proyectos abiertos la información de cada hallazgo (el recurso afectado, su gravedad, quién lo corrige y qué se hizo) queda repartida entre la planilla y los mensajes. La gravedad depende del criterio de quien anota y no queda constancia de cuándo cambió el estado ni por qué, así que cuando un referente pregunta por su proyecto la respuesta se arma a mano revisando las dos fuentes.

A eso se suma un problema de acceso. La planilla es un único archivo y compartirla con un referente implica mostrarle también los hallazgos de otros proyectos (un servidor mal configurado de un proyecto ajeno es justo el tipo de dato que no debería circular fuera del área). La aplicación resuelve las dos cuestiones con un mismo registro, porque cada hallazgo guarda sus datos, una severidad calculada con un método estándar y cada cambio de estado con su comentario, y sobre ese registro el acceso por proyecto deja que el cliente consulte lo suyo sin ver lo ajeno. En los términos de la lectura 1, las entradas son los hallazgos con sus proyectos y activos, el proceso es su registro, evaluación y seguimiento, y las salidas son las consultas y los listados (Universidad Siglo 21, s. f.-a).

Este documento retoma la primera entrega con las correcciones de la devolución (justificación, objetivos, elicitación, procesos de negocio, modelo de dominio y casos de uso) y suma las etapas de análisis, diseño, implementación y pruebas, la base de datos y las definiciones de comunicación.

## **2. Definiciones del proyecto y del sistema**

En esta versión separo el objetivo del proyecto, lo que tiene que hacer el sistema y los fines de la organización a los que ambos aportan, tres cosas que antes aparecían mezcladas.

### **2.1 Objetivo general del proyecto**

Centralizar el seguimiento de las vulnerabilidades de los recursos del área tecnológica de Derechos Del Fútbol para que, antes de diciembre de 2026, cada hallazgo tenga una severidad calculada con un criterio uniforme, un responsable y un historial verificable, y cada referente conozca el estado de sus proyectos sin acceder a información ajena.

### **2.2 Objetivos específicos**

1. Contar con un registro único de hallazgos por proyecto y activo que reemplace a la planilla y a los mensajes.
2. Obtener la severidad de cada hallazgo con CVSS 3.1 base, con resultados iguales a los ejemplos oficiales de FIRST.
3. Conservar la trazabilidad de cada corrección (responsable, fecha objetivo y cada cambio de estado con su motivo).
4. Asegurar que cada cliente consulte solamente los hallazgos de los proyectos que tiene asignados.

### 2.3 Objetivo del sistema y fines de la organización

El sistema tiene que registrar los hallazgos de seguridad de cada proyecto, calcular su severidad CVSS 3.1, controlar su ciclo de estados (PENDIENTE, EN\_TRATAMIENTO y CERRADA, con reapertura) y ofrecer a cada usuario las consultas que corresponden a su rol.

Derechos Del Fútbol trabaja en el ámbito jurídico deportivo y su fin es prestar ese servicio con la información de sus clientes protegida y disponible. El proyecto aporta a ese fin porque acorta el tiempo en que un recurso queda expuesto y deja constancia de cómo se trató cada problema (algo que también sirve ante una auditoría o un reclamo).

### 2.4 Alcance y límites

| Aspecto       | Definición                                                                                                                                                                                                                                                            |
|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Límites       | Desde el alta de un proyecto con su cliente y sus activos hasta el cierre verificado (o la reapertura) de cada hallazgo y la consulta de su historial.                                                                                                                |
| Incluye       | Cuentas con rol administrador o cliente, proyectos, activos y responsables, registro de hallazgos, cálculo CVSS 3.1 base, asignación con fecha objetivo, ciclo de estados con historial, consultas por proyecto, estado y severidad, y exportación del listado a TXT. |
| No contempla  | Escáneres o detección automática, explotación o remediación automática, métricas temporales y ambientales de CVSS, avisos por correo, acceso web o móvil e integración con otros sistemas.                                                                            |
| Restricciones | Java y Swing, base MySQL y aplicación de escritorio en la red interna del área.                                                                                                                                                                                       |

### 2.5 Aplicación del PUD

Aplico el PUD de forma iterativa e incremental, guiada por los casos de uso y centrada en la arquitectura MVC. Cada iteración retoma los artefactos anteriores y los corrige cuando hace falta (por eso este documento integra la definición del proyecto con los modelos nuevos). En el inicio se definieron el problema, los objetivos y el alcance. La elaboración cubre el negocio, los requerimientos, los casos de uso, el análisis, el diseño, el despliegue, las pruebas y la base de datos, que es lo que desarrolla esta entrega. La construcción suma el código Java con la interfaz Swing y la transición es la demostración con el usuario de referencia.

## 3. Elicitación

La elicitación consiste en reunir el conocimiento necesario para producir un modelo de los requerimientos (Loucopoulos, 1995, como se citó en Universidad Siglo 21, s. f.-a). Como formo parte del área combiné la observación del trabajo diario con la revisión de los registros existentes, una entrevista con el responsable del área tecnológica y un cuestionario breve a los referentes de proyecto. Las preguntas y la síntesis de las respuestas están en el Anexo A.

### 3.1 Fuentes de información

| Fuente                                   | Técnica                  | Aporte al proyecto                                                                    |
|------------------------------------------|--------------------------|---------------------------------------------------------------------------------------|
| Planilla de hallazgos y correos del área | Revisión documental      | Qué datos se registran hoy y cuáles faltan (responsable, fecha objetivo e historial). |
| Responsable del área tecnológica         | Entrevista (Anexo A.1)   | Reglas de asignación, cierre y reapertura, y qué puede ver cada persona.              |
| Referentes de proyecto                   | Cuestionario (Anexo A.2) | Qué necesitan consultar y con qué frecuencia.                                         |
| Trabajo propio en el área                | Observación participante | Recorrido real de un hallazgo desde la detección hasta la verificación.               |
| Especificación CVSS 3.1 (FIRST, 2019a)   | Revisión documental      | Métricas base, fórmula, redondeo y categorías de severidad.                           |

### 3.2 Actores e interesados

Los actores son el administrador (personal del área tecnológica que registra, evalúa, asigna y sigue los hallazgos, y que además da de alta las cuentas) y el cliente (referente de un proyecto, que solo consulta). El responsable de corrección es la persona o el equipo que corrige sobre el activo (soporte interno, desarrollo o un proveedor) y no usa el sistema (es un dato del catálogo). La dirección también es interesada porque le importa el nivel de exposición de la organización, aunque no opera el sistema y recibe el listado exportado.

### 3.3 Necesidades y requerimientos iniciales

| Necesidad detectada                                    | Requerimiento inicial                                            | RF / RNF                 |
|--------------------------------------------------------|------------------------------------------------------------------|--------------------------|
| Tener en un solo lugar todos los datos de un hallazgo. | Registrar hallazgos por proyecto y activo con sus datos mínimos. | RF01, RF03, RF09         |
| Priorizar con un criterio común.                       | Calcular la severidad con CVSS 3.1 base.                         | RF12                     |
| Saber quién corrige y para cuándo.                     | Asignar responsable y fecha objetivo.                            | RF02, RF04               |
| Reconstruir qué pasó con cada hallazgo.                | Guardar cada cambio de estado con fecha, usuario y comentario.   | RF06, RF07, RNF03        |
| Que el referente vea su avance sin pedirlo.            | Consultas filtradas con acceso por proyecto.                     | RF05, RNF04              |
| Que un referente no vea hallazgos ajenos.              | Cuentas con rol y control de acceso en cada consulta.            | RF10, RF11, RNF04, RNF05 |
| Entregar un resumen a la dirección.                    | Exportar el listado filtrado.                                    | RF08                     |

## 4. Conocimiento del negocio

Derechos Del Fútbol trabaja en el ámbito jurídico deportivo. El área tecnológica administra sus recursos informáticos (el sitio web, la aplicación interna de expedientes, las bases de datos, la red de la oficina y los respaldos) y el proceso que el proyecto informatiza es el seguimiento de los hallazgos de seguridad sobre esos recursos.

### 4.1 Proceso de negocio actual

El proceso relevado es el seguimiento de un hallazgo tal como se hace hoy, con tres roles (administrador del área, responsable de corrección y referente del proyecto). La figura 1 lo muestra como diagrama de actividades de UML con un carril por rol.

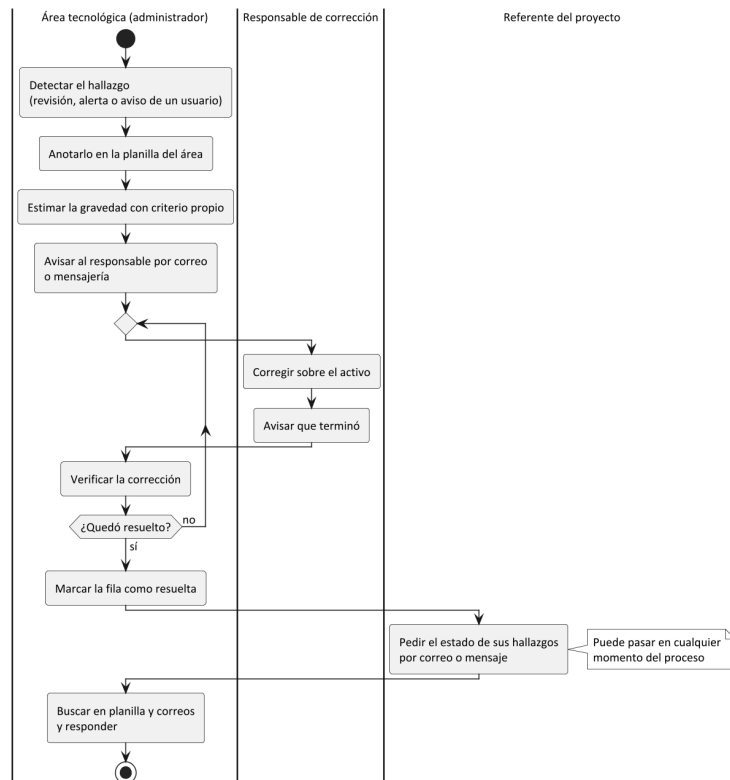


Figura 1. Proceso actual de seguimiento de un hallazgo. Elaboración propia.

El hallazgo aparece por una revisión propia, una alerta de un equipo o el aviso de un usuario. El administrador lo anota en la planilla con su propio criterio de gravedad y avisa al responsable, y el ciclo de corrección y verificación se repite hasta que el problema queda resuelto. En paralelo el referente pregunta cómo va su proyecto y la respuesta sale de revisar planilla y correos. De ese recorrido salen los problemas que el sistema tiene que resolver. Hallazgos parecidos reciben gravedades distintas porque no hay un método común, no queda registro de cuándo cambió el estado ni por qué, un hallazgo cerrado que reaparece se anota como nuevo (y se pierde su historia) y compartir la planilla expone hallazgos de otros proyectos.

## 4.2 Modelo de dominio

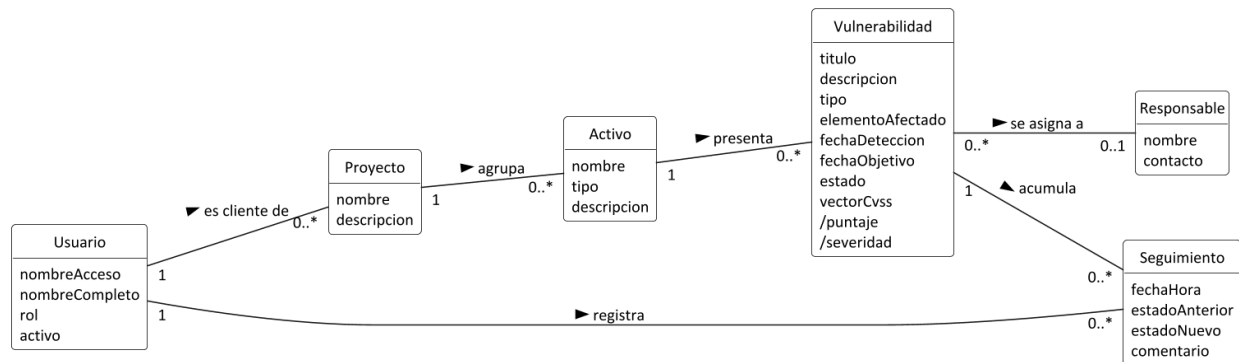


Figura 2. Modelo de dominio con clases conceptuales, atributos y multiplicidades. Elaboración propia.

En esta versión rehízo el modelo con la notación de clases conceptuales de UML (nombre y atributos, sin tipos ni métodos, porque todavía describe el negocio y no el software) y agregué la asociación entre Usuario y Seguimiento para saber quién registró cada cambio. La barra en /puntaje y /severidad marca atributos derivados, que se calculan a partir del vector CVSS y no se cargan. Un cliente puede tener varios proyectos y cada proyecto tiene un solo cliente. El proyecto de un hallazgo se obtiene a través de su activo, por eso un activo con hallazgos no puede pasar a otro proyecto (cambiaría quién ve esa información).

## 5. Propuesta de solución

### 5.1 Propuesta funcional

La propuesta sigue el recorrido de un hallazgo, desde la preparación de la estructura hasta la consulta del resultado, y la tabla ordena las funciones en ese mismo orden.

| Momento               | Funciones del sistema                                                                                                       | CU                     |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------|------------------------|
| <b>1. Preparación</b> | El administrador crea las cuentas, los proyectos con su cliente, los activos y el catálogo de responsables.                 | CU10, CU09, CU01, CU02 |
| <b>2. Registro</b>    | Alta del hallazgo en estado PENDIENTE. La severidad se calcula en el mismo formulario o queda «Sin evaluar» para después.   | CU03, CU12             |
| <b>3. Tratamiento</b> | Asignación de responsable y fecha objetivo y cambios de estado con comentario, que quedan en el historial.                  | CU04, CU06, CU07       |
| <b>4. Consulta</b>    | Listados por proyecto, estado y severidad. El cliente solo ve sus proyectos y el administrador puede exportar el resultado. | CU05, CU13, CU08       |
| <b>Transversal</b>    | Inicio y cierre de sesión, que habilita las funciones según el rol.                                                         | CU11                   |

El administrador usa todas las funciones y el cliente tiene tres (iniciar sesión, consultar sus hallazgos y ver el historial). Ninguna consulta del cliente muestra datos de un proyecto que no le pertenece, aunque conozca el número de un hallazgo ajeno.

### 5.2 Propuesta técnica

La aplicación es de escritorio y sigue el patrón modelo-vista-controlador (MVC) con una capa DAO para el acceso a datos. Las vistas Swing muestran y capturan datos, los controladores coordinan cada caso de uso, las clases del modelo aplican las reglas (transiciones, fechas y cálculo CVSS) y los DAO ejecutan las consultas con JDBC. Tener el acceso a datos aparte deja cambiar la ubicación de la base sin tocar las pantallas.

| Componente      | Elección | Motivo                                                     |
|-----------------|----------|------------------------------------------------------------|
| <b>Lenguaje</b> | Java 21  | Lo pide la materia y es una versión con soporte extendido. |

| Componente     | Elección                                        | Motivo                                                                          |
|----------------|-------------------------------------------------|---------------------------------------------------------------------------------|
| Interfaz       | Swing                                           | Viene incluida en el JDK y no necesita librerías externas.                      |
| Base de datos  | MySQL de XAMPP (MariaDB 10.4)                   | La pide la materia y tiene claves foráneas y transacciones (motor InnoDB).      |
| Acceso a datos | JDBC con MySQL Connector/J                      | Es la API estándar de Java y las sentencias preparadas evitan la inyección SQL. |
| IDE            | Apache NetBeans                                 | Tiene diseñador de formularios Swing y depurador.                               |
| Diagramas      | PlantUML                                        | Los diagramas quedan guardados como texto en el repositorio.                    |
| Versionado     | Git y GitHub                                    | Historial de cambios y publicación del código.                                  |
| Hardware       | PC con Windows 10 u 11, 8 GB de RAM y red local | Alcanza para el JRE y el servidor de base en el mismo equipo del prototipo.     |

## 6. Requerimientos

### 6.1 Requerimientos funcionales

Cada requerimiento se corresponde con el caso de uso del mismo número.

| RF / CU                                       | Descripción                                                                                                                                                                     |
|-----------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>RF01 / CU01 Gestionar activos</b>          | El administrador da de alta, modifica y lista activos de un proyecto. Solo puede borrar o trasladar un activo que no tenga vulnerabilidades.                                    |
| <b>RF02 / CU02 Gestionar responsables</b>     | El administrador da de alta, modifica y lista responsables. Solo puede borrar uno que no tenga vulnerabilidades asignadas.                                                      |
| <b>RF03 / CU03 Registrar vulnerabilidad</b>   | El administrador registra título, descripción, tipo, elemento afectado, activo y fecha de detección. El sistema asigna un número y el estado PENDIENTE.                         |
| <b>RF04 / CU04 Asignar responsable</b>        | El administrador asigna un responsable y una fecha objetivo igual o posterior a la detección, y puede cambiarlos mientras el hallazgo no esté cerrado.                          |
| <b>RF05 / CU05 Consultar vulnerabilidades</b> | El administrador consulta todos los proyectos y el cliente solo los suyos, con filtros de proyecto, estado y severidad (incluida «Sin evaluar»).                                |
| <b>RF06 / CU06 Cambiar estado</b>             | El administrador inicia el tratamiento, cierra o reabre un hallazgo. El sistema valida la transición, exige un comentario y guarda estado y seguimiento en una misma operación. |
| <b>RF07 / CU07 Consultar historial</b>        | El usuario consulta fecha y hora, estados anterior y nuevo, autor y comentario de cada cambio, con los mismos permisos por proyecto que en CU05.                                |
| <b>RF08 / CU08 Exportar listado</b>           | El administrador guarda el resultado filtrado en un archivo TXT nuevo. El sistema informa si se guardó o si hubo un error.                                                      |
| <b>RF09 / CU09 Gestionar proyectos</b>        | El administrador crea y modifica proyectos y les asigna una cuenta cliente activa. Solo puede borrar un proyecto sin activos.                                                   |
| <b>RF10 / CU10 Gestionar usuarios</b>         | El administrador crea cuentas con nombre de acceso único, contraseña y rol, y puede cambiar contraseñas o desactivar cuentas (salvo al último administrador activo).            |
| <b>RF11 / CU11 Iniciar y cerrar sesión</b>    | El sistema valida las credenciales de una cuenta activa, habilita las operaciones de su rol y al cerrar elimina la sesión y los datos visibles.                                 |
| <b>RF12 / CU12 Calcular CVSS</b>              | El administrador selecciona las ocho métricas base de CVSS 3.1 y el sistema calcula puntaje y severidad, los guarda con el hallazgo y deja recalcular.                          |

Las vulnerabilidades no se borran sino que se conservan para consultar su historial, y pasan a CERRADA cuando se resuelven. La severidad se informa como ninguna (0,0), baja (0,1 a 3,9), media (4,0 a 6,9), alta (7,0 a 8,9) o crítica (9,0 a 10,0) según FIRST (2019a).

### 6.2 Requerimientos no funcionales

| ID                        | Requerimiento y criterio de aceptación                                                                                                                                                                                      |
|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| RNF01 Plataforma          | Aplicación de escritorio en Java 21 y Swing, con persistencia en MySQL mediante JDBC, para Windows 10 u 11.                                                                                                                 |
| RNF02 Integridad          | Se rechazan campos obligatorios vacíos, referencias inexistentes, fechas objetivo anteriores a la detección y transiciones inválidas.                                                                                       |
| RNF03 Atomicidad          | El cambio de estado y su seguimiento se guardan en la misma transacción. Si falla una escritura se revierte todo y el hallazgo conserva su estado.                                                                          |
| RNF04 Acceso              | Toda operación exige sesión y verifica rol y proyecto. Un cliente no puede consultar un hallazgo ajeno aunque conozca su número.                                                                                            |
| RNF05 Contraseñas         | Se guardan con hash SHA-256 y una sal distinta por usuario, nunca en texto plano.                                                                                                                                           |
| RNF06 Tiempo de respuesta | Con hasta 5.000 hallazgos cargados, las consultas de CU05 y CU07 responden en menos de 2 segundos en el equipo de referencia.                                                                                               |
| RNF07 Usabilidad          | Interfaz en español, campos obligatorios marcados, mensajes que dicen qué corregir y acciones visibles según el rol. Un usuario nuevo registra un hallazgo en menos de 3 minutos después de una demostración de 10 minutos. |
| RNF08 Normas              | Los datos personales de las cuentas se tratan según la Ley 25.326 (datos mínimos, acceso restringido y confidencialidad) y la severidad sigue la especificación CVSS 3.1 de FIRST.                                          |

## 7. Casos de uso

### 7.1 Diagrama de casos de uso

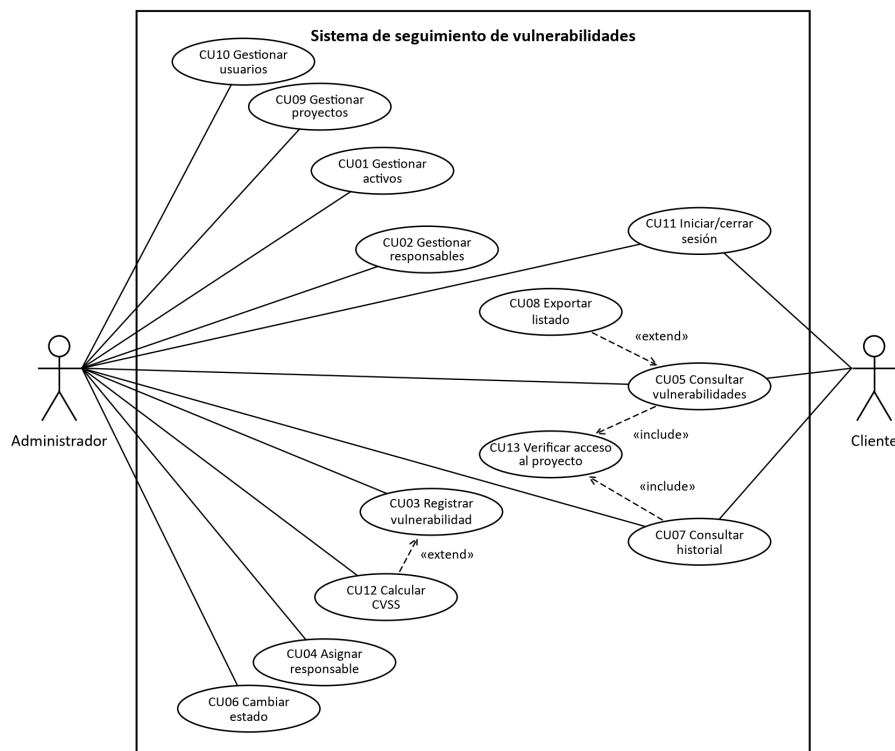


Figura 3. Diagrama de casos de uso con el límite del sistema. Elaboración propia.

El rectángulo marca el límite del sistema y deja a los actores afuera. Respecto de la entrega anterior ordené el diagrama por actor (los casos del administrador a la izquierda y los compartidos junto al cliente) para que no se lea como una secuencia, y usé los estereotipos según su significado. «include» indica un comportamiento obligatorio que forma parte del caso base, por eso CU05 y CU07 incluyen CU13 (ninguna consulta puede mostrar datos sin verificar antes a qué proyectos accede la cuenta, y como la regla es la misma en los dos casos se escribe una sola vez). «extend» indica un comportamiento opcional que se agrega en un punto del caso base. CU12 extiende CU03 porque un hallazgo puede registrarse sin evaluar, y CU08 extiende CU05 porque la exportación es optativa y solo la usa el administrador. El inicio de sesión (CU11) no se modela como «include» de cada caso porque es una precondition de todos y no un paso de su flujo.

### 7.2 Especificación de los casos críticos

Detallo CU03, CU06 y CU11 porque cubren el registro, el seguimiento y el acceso, que son los puntos donde un error tiene más consecuencias (un cambio de estado sin historial o una sesión mal cerrada en un equipo compartido).

|                            |                                                                                                                                                                                                                                                                                                                                                                                                     |
|----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Caso de uso</b>         | <b>CU03 Registrar vulnerabilidad</b>                                                                                                                                                                                                                                                                                                                                                                |
| <b>Actor</b>               | Administrador.                                                                                                                                                                                                                                                                                                                                                                                      |
| <b>Objetivo</b>            | Incorporar un hallazgo asociado a un activo de un proyecto.                                                                                                                                                                                                                                                                                                                                         |
| <b>Requerimientos</b>      | RF03, RNF02 y RNF04.                                                                                                                                                                                                                                                                                                                                                                                |
| <b>Precondiciones</b>      | Sesión de administrador activa (CU11). Existen el proyecto y el activo.                                                                                                                                                                                                                                                                                                                             |
| <b>Flujo principal</b>     | 1. El administrador elige «Nuevo hallazgo». 2. Selecciona el proyecto y el activo. 3. Completa título, descripción, tipo, elemento afectado y fecha de detección. 4. Punto de extensión «evaluar severidad». 5. Confirma el alta. 6. El sistema valida los campos obligatorios, la fecha y que el activo pertenezca al proyecto. 7. Guarda el hallazgo con estado PENDIENTE. 8. Muestra el detalle. |
| <b>Flujos alternativos</b> | 4a. Si el administrador completa las métricas se ejecuta CU12 y el vector queda guardado con el hallazgo. 6a. Si falta un dato o la fecha es inválida, el sistema indica qué campo corregir y vuelve al paso 3. 7a. Si falla la escritura informa el error y no se crea el hallazgo.                                                                                                                |
| <b>Postcondiciones</b>     | El hallazgo existe en estado PENDIENTE, sin responsable ni fecha objetivo, evaluado o «Sin evaluar», y con el historial vacío.                                                                                                                                                                                                                                                                      |

|                            |                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Caso de uso</b>         | <b>CU06 Cambiar estado</b>                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <b>Actor</b>               | Administrador.                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| <b>Objetivo</b>            | Mover un hallazgo en su ciclo de estados y dejar constancia del motivo.                                                                                                                                                                                                                                                                                                                                                                                                 |
| <b>Requerimientos</b>      | RF06 (relacionado con RF04 y RF07), RNF02 y RNF03.                                                                                                                                                                                                                                                                                                                                                                                                                      |
| <b>Precondiciones</b>      | Sesión de administrador activa y hallazgo existente. Para iniciar el tratamiento el hallazgo tiene responsable y fecha objetivo (CU04).                                                                                                                                                                                                                                                                                                                                 |
| <b>Flujo principal</b>     | 1. El administrador selecciona un hallazgo y elige «Cambiar estado». 2. El sistema muestra el estado actual y solo las transiciones permitidas. 3. El administrador elige el nuevo estado y escribe el comentario. 4. El sistema valida la transición, la asignación y el comentario. 5. En una sola transacción actualiza el estado y registra el seguimiento (fecha y hora, usuario, estado anterior, estado nuevo y comentario). 6. Confirma y actualiza el listado. |
| <b>Flujos alternativos</b> | 4a. Si la transición no está permitida (por ejemplo, cerrar desde PENDIENTE) se rechaza sin cambios. 4b. Si falta responsable o fecha objetivo se rechaza y se indica ejecutar CU04. 4c. Si el comentario está vacío se pide completarlo. 5a. Si falla una escritura se revierte la transacción y el hallazgo conserva su estado.                                                                                                                                       |
| <b>Postcondiciones</b>     | El estado actual coincide con el estado nuevo del último seguimiento.                                                                                                                                                                                                                                                                                                                                                                                                   |

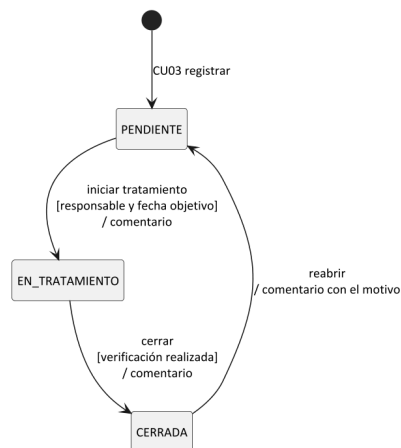


Figura 4. Estados de una vulnerabilidad y transiciones admitidas (CU06). Elaboración propia.

|                            |                                                                                                                                                                                                                                                                                                                                                               |
|----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Caso de uso</b>         | <b>CU11 Iniciar y cerrar sesión</b>                                                                                                                                                                                                                                                                                                                           |
| <b>Actores</b>             | Administrador y cliente.                                                                                                                                                                                                                                                                                                                                      |
| <b>Objetivo</b>            | Acceder con los permisos de la cuenta y cerrar el acceso al terminar.                                                                                                                                                                                                                                                                                         |
| <b>Requerimientos</b>      | RF11, RNF04 y RNF05.                                                                                                                                                                                                                                                                                                                                          |
| <b>Precondiciones</b>      | La cuenta existe y está activa.                                                                                                                                                                                                                                                                                                                               |
| <b>Flujo principal</b>     | 1. El usuario ingresa nombre de acceso y contraseña. 2. El sistema busca la cuenta y calcula el hash de la contraseña con la sal guardada. 3. Si coincide, abre la sesión con el rol de la cuenta. 4. Muestra la consulta con las acciones de ese rol. 5. Al elegir «Cerrar sesión» descarta la sesión, limpia los listados y vuelve a la pantalla de acceso. |
| <b>Flujos alternativos</b> | 2a. Si no hay conexión con la base se informa y no se abre la sesión. 3a. Si las credenciales son incorrectas o la cuenta está inactiva se muestra «Usuario o contraseña incorrectos» (el mismo mensaje en los dos casos, para no revelar qué cuentas existen).                                                                                               |
| <b>Postcondiciones</b>     | Con sesión el usuario opera según su rol y sin sesión no hay operaciones disponibles.                                                                                                                                                                                                                                                                         |

## 8. Etapa de análisis

En el análisis los casos de uso se refinan en clases conceptuales con tres estereotipos. «boundary» representa la interacción con el actor, «control» coordina el caso de uso y «entity» guarda la información que persiste. Todavía no son clases de software (esas aparecen en el diseño) sino el paso intermedio entre los casos de uso y el diagrama de clases.

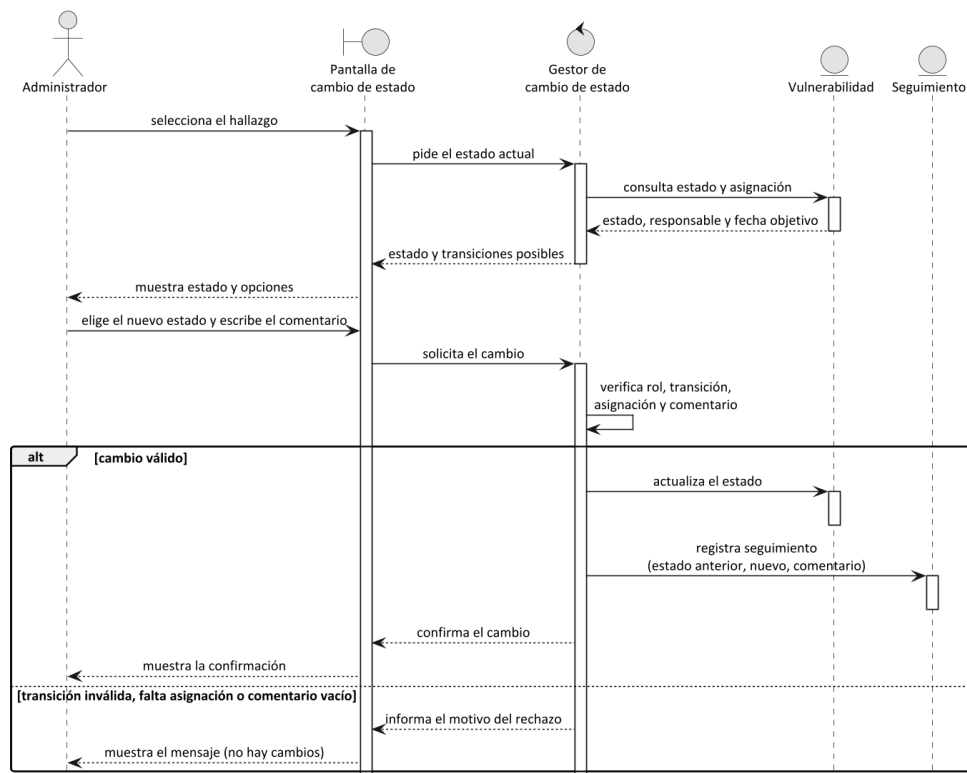


Figura 5. Diagrama de secuencia de análisis de CU06, curso normal y alternativo. Elaboración propia.

Elegí CU06 porque es el caso con más reglas y el que sostiene la trazabilidad. La pantalla solo recibe y muestra datos, y el gestor concentra las validaciones y es el único que modifica las entidades. El fragmento alt separa el curso normal, donde el estado y el seguimiento se escriben juntos, del curso alternativo, donde el gestor informa el motivo del rechazo y ninguna entidad cambia.

## 9. Etapa de diseño

En el diseño las clases de análisis pasan a ser clases de Java con atributos, métodos y tipos. Los atributos coinciden con las columnas de la base (int para las claves, LocalDate para las fechas y un enum para el estado), así los DAO convierten filas en objetos sin transformaciones intermedias. Omití los métodos get y set para que el diagrama se lea mejor.

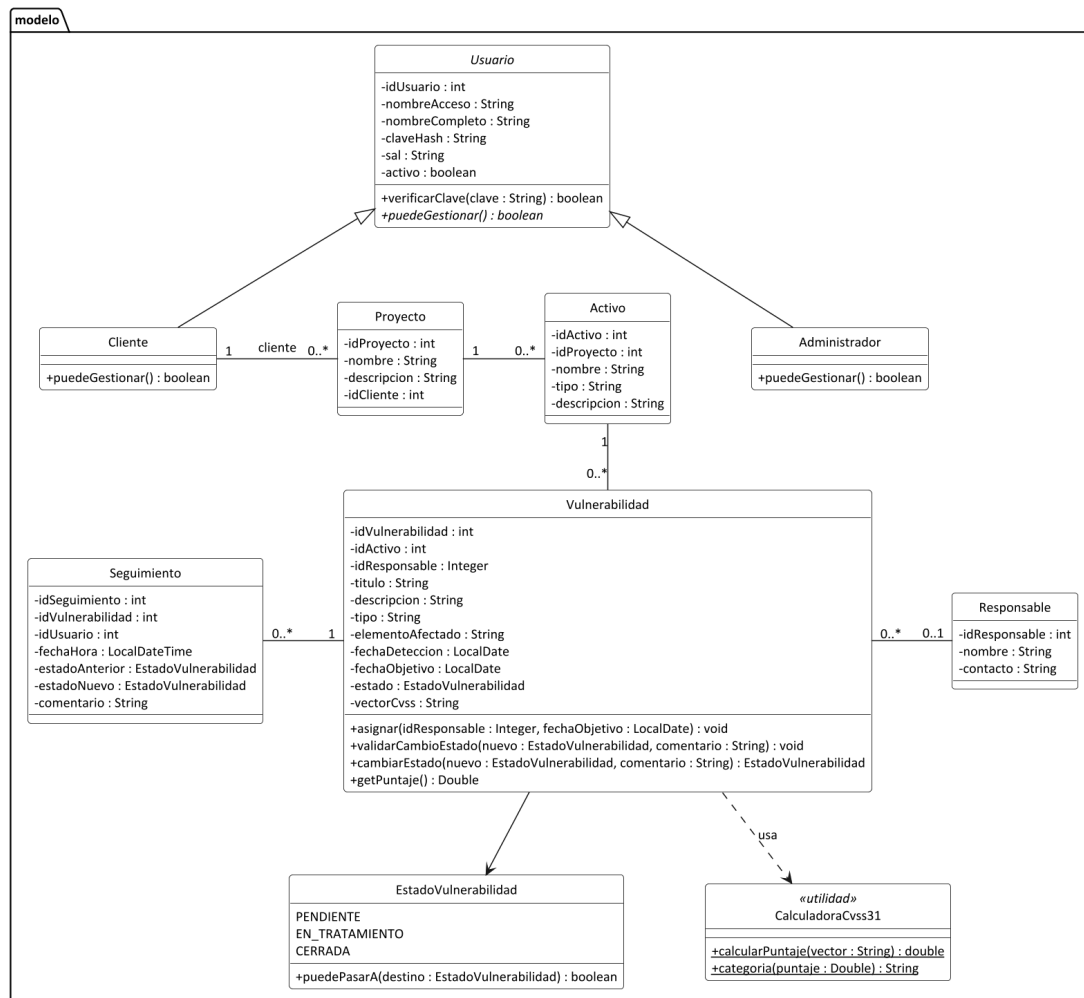


Figura 6. Diagrama de clases de diseño del paquete modelo. Elaboración propia.

Usuario es abstracta y tiene dos subclases, Administrador y Cliente, que responden distinto a puedeGestionar() (la columna rol de la base decide qué subclase se crea al iniciar sesión). Vulnerabilidad concentra las reglas del ciclo de estados y EstadoVulnerabilidad define las tres transiciones permitidas. idResponsable es Integer y no int porque puede ser nulo. CalculadoraCvss31 es una clase utilitaria con métodos estáticos que aplica la fórmula de la especificación a partir del vector, y getPuntaje() la usa para devolver el puntaje (nulo si el hallazgo no está evaluado). Las vistas Swing, los controladores y los DAO completan la arquitectura MVC y se desarrollan en las próximas entregas.

## 10. Etapa de implementación

La lectura 2 define el modelo de despliegue como la distribución física del sistema entre los nodos de cómputo (Universidad Siglo 21, s. f.-b). Las decisiones de despliegue salen de los requerimientos no funcionales.

| RNF   | Decisión de despliegue                                                                 |
|-------|----------------------------------------------------------------------------------------|
| RNF01 | JRE 21 en cada PC con Windows. La aplicación se distribuye como un único archivo .jar. |
| RNF03 | Base MySQL con motor InnoDB, que maneja transacciones y claves foráneas.               |

| RNF           | Decisión de despliegue                                                                                                       |
|---------------|------------------------------------------------------------------------------------------------------------------------------|
| RNF04 y RNF08 | La base queda en un equipo de la red interna, sin salida a Internet, y el puerto 3306 solo se habilita para las PC del área. |
| RNF06         | El servidor de base está en la misma red local que los puestos que consultan, sin pasar por Internet.                        |

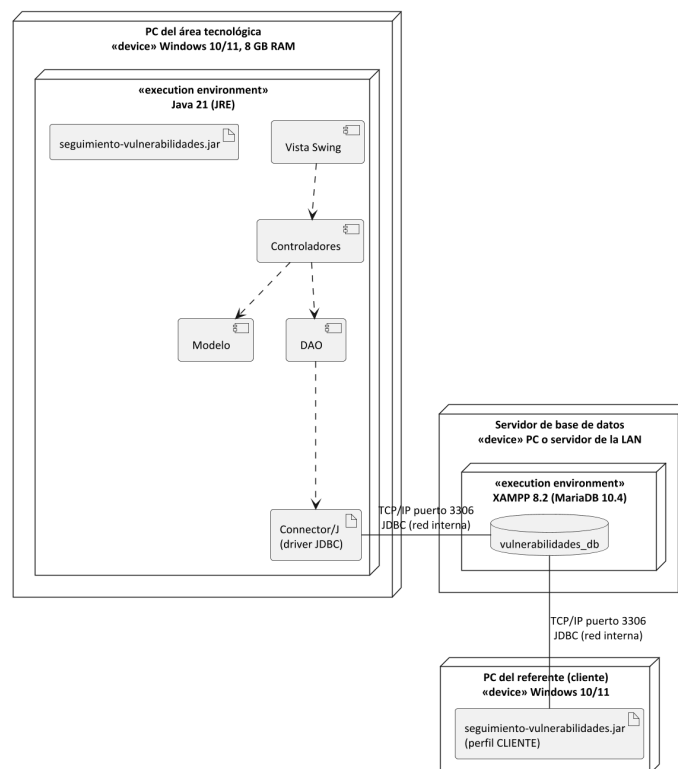


Figura 7. Diagrama de despliegue. Elaboración propia.

En el prototipo los tres nodos son el mismo equipo y la aplicación se conecta a localhost. En la operación real la base queda en un equipo del área y las PC del administrador y de los referentes ejecutan el mismo .jar (lo único que cambia es el rol de la cuenta que inicia sesión). Hay un riesgo que dejo documentado. Con una conexión directa a la base, cada PC que ejecuta la aplicación tiene la cadena de conexión, por eso el acceso por proyecto se controla también en las consultas y en producción se usaría un usuario de base con permisos mínimos (en el prototipo se usa el usuario de XAMPP).

## 11. Etapa de pruebas

La lectura 2 organiza las pruebas en unitarias (de caja blanca y de caja negra), de integración, de sistema y de aceptación, y aclara que buscan que el software falle antes de que lo haga frente al usuario (Universidad Siglo 21, s. f.-b).

### 11.1 Plan de pruebas

| CU         | Código | Tipo        | Técnica                        | Observaciones                                                  | Estado      |
|------------|--------|-------------|--------------------------------|----------------------------------------------------------------|-------------|
| CU12       | CP01   | Unitaria    | Caja negra, valores límite     | CalculadoraCvss31.categoria()                                  | Ejecutada   |
| CU12       | CP02   | Unitaria    | Caja negra, ejemplos oficiales | calcularPuntaje() con tres vectores de FIRST                   | Ejecutada   |
| CU06       | CP03   | Unitaria    | Caja blanca, caminos básicos   | Vulnerabilidad.validarCambioEstado()                           | Ejecutada   |
| CU03, CU04 | CP04   | Integración | Caja negra, datos inválidos    | CHECK de fechas y clave foránea de la base                     | Ejecutada   |
| CU05, CU13 | CP05   | Sistema     | Funcional                      | Un cliente no ve hallazgos ajenos desde la interfaz            | Planificada |
| Todos      | CP06   | Aceptación  | Validación con el usuario      | Recorrido completo de un hallazgo con el usuario de referencia | Planificada |

### 11.2 Caso de prueba de caja negra (CP01)

Verifica RF12 sobre `CalculadoraCvss31.categoria()`, que recibe el puntaje y devuelve la severidad. Las clases válidas son null (sin evaluar), 0,0 (ninguna), 0,1 a 3,9 (baja), 4,0 a 6,9 (media), 7,0 a 8,9 (alta) y 9,0 a 10,0 (crítica), y las inválidas son los puntajes menores que 0,0 o mayores que 10,0. Sobre cada frontera tomé un valor de cada lado (análisis de valores límite).

| Entrada | Esperado    | Obtenido    | Entrada | Esperado | Obtenido |
|---------|-------------|-------------|---------|----------|----------|
| null    | Sin evaluar | Sin evaluar | 6,9     | Media    | Media    |
| -0,1    | Rechazo     | Rechazo     | 7,0     | Alta     | Alta     |
| 0,0     | Ninguna     | Ninguna     | 8,9     | Alta     | Alta     |
| 0,1     | Baja        | Baja        | 9,0     | Crítica  | Crítica  |
| 3,9     | Baja        | Baja        | 10,0    | Crítica  | Crítica  |
| 4,0     | Media       | Media       | 10,1    | Rechazo  | Rechazo  |

Los 12 casos dieron el resultado esperado (`salida_pruebas_java.txt` del repositorio). CP02 completa la prueba del cálculo comparando `calcularPuntaje()` con tres vectores publicados por FIRST (2019b), 6,1 para CVE-2013-1937, 7,5 para Heartbleed y 9,8 para Shellshock, y los tres coincidieron.

### 11.3 Caso de prueba de caja blanca (CP03)

Verifica el método `validarCambioEstado()` de `Vulnerabilidad`, que tiene tres decisiones (D1 transición no permitida, D2 inicio del tratamiento sin asignación y D3 comentario vacío). El grafo de flujo tiene 9 nodos y 11 aristas, así que la complejidad ciclomática es  $V(G) = 11 - 9 + 2 = 4$  y hay cuatro caminos independientes.

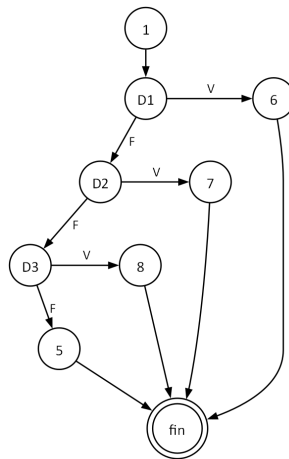


Figura 8. Grafo de flujo de validarCambioEstado(). Elaboración propia.

| Camino | Recorrido             | Datos de entrada                | Esperado             | Obtenido       |
|--------|-----------------------|---------------------------------|----------------------|----------------|
| C1     | 1, D1, 6, fin         | PENDIENTE a CERRADA             | Rechazo (transición) | Rechazo        |
| C2     | 1, D1, D2, 7, fin     | Inicio sin responsable ni fecha | Rechazo (asignación) | Rechazo        |
| C3     | 1, D1, D2, D3, 8, fin | Asignado, comentario en blanco  | Rechazo (comentario) | Rechazo        |
| C4     | 1, D1, D2, D3, 5, fin | Asignado, comentario válido     | EN_TRATAMIENTO       | EN_TRATAMIENTO |

#### 11.4 Procedimiento de prueba

1. Compilar el modelo y las pruebas con el JDK 21 desde la carpeta TP2/pruebas del repositorio (javac -encoding UTF-8 -d bin src/modelo/\*.java src/pruebas/\*.java).
2. Ejecutar java -cp bin pruebas.PruebasTP2 y guardar la salida. Cada caso imprime PASA o FALLA con el valor esperado y el obtenido.
3. Recrear la base con los scripts 01 a 04 y revisar que la base rechace los dos datos inválidos de CP04.
4. Si un resultado no coincide, registrar el defecto con la plantilla de 11.5 y, después de corregirlo, repetir las pruebas afectadas antes de cerrarlo.

#### 11.5 Tratamiento de defectos

Cada defecto sigue el ciclo abierto, asignado, corregido, verificado y cerrado, y si la verificación falla vuelve a asignado (es el mismo ciclo que el sistema propone para los hallazgos). Como las pruebas ejecutadas en esta etapa no encontraron defectos, la plantilla se muestra completa con un defecto de ejemplo.

| Campo            | Registro                                                                     |
|------------------|------------------------------------------------------------------------------|
| Identificador    | DEF-01 (ejemplo)                                                             |
| Prueba de origen | CP01, caso categoria(0.0)                                                    |
| Descripción      | Devuelve «Baja» cuando el resultado esperado es «Ninguna».                   |
| Severidad        | Media (un hallazgo sin impacto aparecería como de riesgo bajo).              |
| Causa            | La comparación puntaje < 4.0 se evalúa antes que el caso 0,0.                |
| Acción           | Evaluar primero el caso 0,0 y después los rangos.                            |
| Verificación     | Repetir CP01 completo (regresión) y cerrar el defecto si pasan los 12 casos. |
| Estado           | Cerrado                                                                      |

## 11.6 Evaluación de las pruebas

| Prueba       | Casos | Aprobados | Fallidos | Observación                                                                 |
|--------------|-------|-----------|----------|-----------------------------------------------------------------------------|
| CP01         | 12    | 12        | 0        | Todas las fronteras de las categorías de severidad.                         |
| CP02         | 3     | 3         | 0        | Coincide con los ejemplos oficiales de FIRST.                               |
| CP03         | 4     | 4         | 0        | Cubre los cuatro caminos independientes.                                    |
| CP04         | 2     | 2         | 0        | La base rechazó la fecha inválida (4025) y el borrado con hallazgos (1451). |
| <b>Total</b> | 21    | 21        | 0        | Sin defectos en esta etapa.                                                 |

Los casos ejecutados pasaron, pero el resultado todavía no dice nada sobre la interfaz ni sobre el rendimiento. Por eso CP05 y CP06 quedan para la construcción, cuando la aplicación Swing esté integrada con la base.

## 12. Prototipos de interfaz

Los prototipos son ventanas Swing reales (el mismo toolkit del sistema) generadas por el programa PrototiposInterfaz del repositorio. No tienen lógica conectada, salvo el panel CVSS, que usa la clase CalculadoraCvss31. Siguiendo a Kendall y Kendall (2011), sirven para mostrar cómo se va a ver la herramienta antes de construirla, y cada uno está vinculado al caso de uso que implementa.

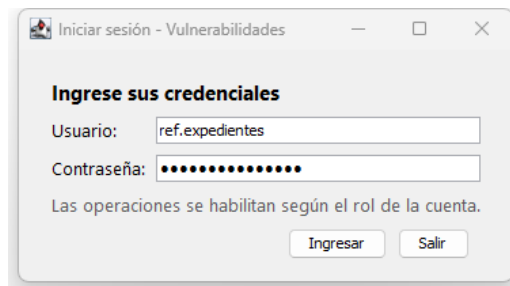


Figura 9. Prototipo de inicio de sesión (CU11). Elaboración propia.

Es la pantalla inicial de CU11, con nombre de acceso y contraseña enmascarada.

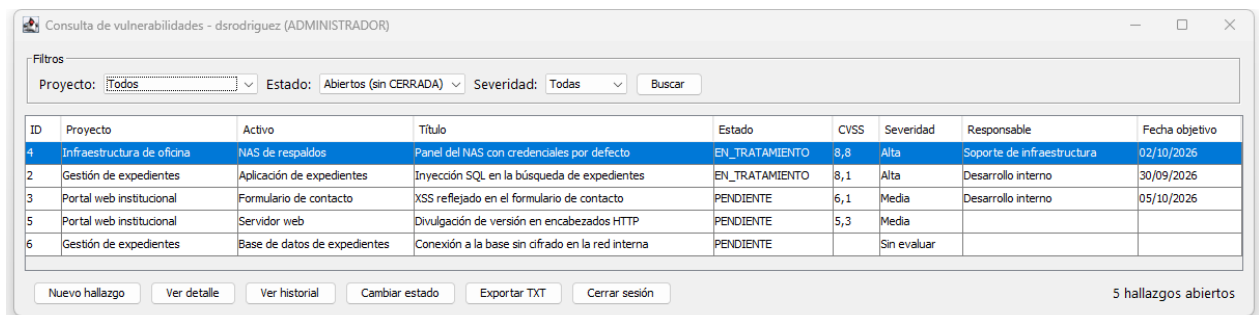


Figura 10. Prototipo de consulta de vulnerabilidades (CU05). Elaboración propia.

Es la pantalla principal. Los filtros son los de RF05 y los botones abren el registro, el historial, el cambio de estado y la exportación. Para una cuenta cliente se ocultan los botones de gestión y el filtro de proyecto solo lista sus proyectos (CU13).

**Registrar vulnerabilidad**

**Datos del hallazgo**

Proyecto:

Activo:

Título:

Tipo:

Elemento afectado:

Fecha de detección:

Descripción:

**Evaluación CVSS 3.1 (opcional, CU12)**

Vector de ataque (AV):

Complejidad (AC):

Privilegios (PR):

Interacción (UI):

Alcance (S):

Confidencialidad (C):

Integridad (I):

Disponibilidad (A):

**Puntaje: 8,1 Severidad: Alta**

CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N

Figura 11. Prototipo de registro de vulnerabilidad con el panel CVSS (CU03 extendido por CU12). Elaboración propia.

El formulario de la izquierda es CU03 y el panel de la derecha es la extensión CU12. Si no se completan las métricas el hallazgo queda «Sin evaluar». El puntaje que se ve (8,1, alta) lo calculó la clase real.

### 13. Base de datos

#### 13.1 Definición y criterios de diseño

La base se llama vulnerabilidades\_db y corre en el MySQL de XAMPP (MariaDB 10.4, compatible con MySQL). El criterio principal fue evitar la redundancia (Reinosa et al., 2012, como se citó en Universidad Siglo 21, s. f.-b) sin complicar el esquema. Quedaron seis tablas con claves primarias numéricas autoincrementales, una clave foránea por cada relación, UNIQUE en los datos que no se pueden repetir (nombre de acceso y nombre de proyecto), ENUM para el rol y el estado, y una restricción CHECK para que la fecha objetivo no sea anterior a la detección. Las reglas más complejas (transiciones de estado y permisos por proyecto) se controlan en Java. El motor es InnoDB porque maneja transacciones y claves foráneas, y el juego de caracteres utf8mb4 guarda sin problemas tildes y eñes.

#### 13.2 Normalización

Todos los atributos son atómicos y todas las claves primarias son simples, por lo que las tablas están en primera y segunda forma normal. Para la tercera forma normal ningún atributo no clave depende de otro atributo no clave, y por eso el puntaje y la severidad no se guardan (dependen del vector CVSS y se calculan en Java cada vez que hacen falta, así nunca quedan desactualizados). El vector se guarda como una sola cadena porque es el formato estándar de FIRST y la aplicación siempre lo lee y lo escribe completo.

### 13.3 Diagrama entidad-relación

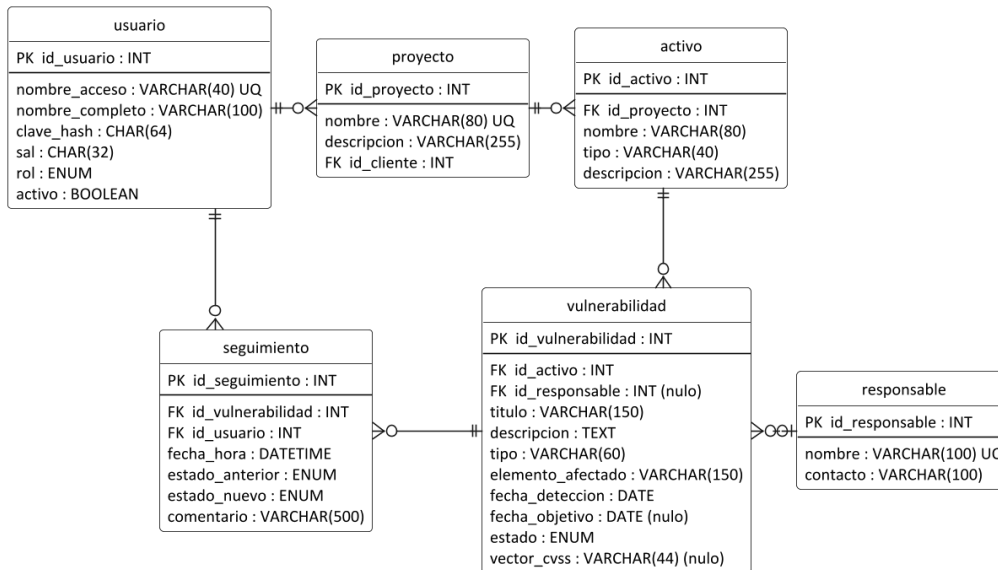


Figura 12. Diagrama entidad-relación (notación pata de gallo) con claves y tipos. Elaboración propia.

En la notación pata de gallo la raya indica uno, el círculo cero (opcional) y la pata muchos. Un hallazgo tiene cero o un responsable (id\_responsable admite nulo) y un responsable puede tener muchos hallazgos. El usuario aparece en dos relaciones, como cliente de proyectos y como autor de seguimientos.

### 13.4 Creación de la base y de las tablas

El script 01\_creacion.sql crea la base con utf8mb4 y las seis tablas. La captura muestra el resultado en phpMyAdmin, todavía sin registros.

| Tabla                                   | Acción                                              | Filas    | Tipo          | Cotejamiento              | Tamaño          | Residuo a depurar |
|-----------------------------------------|-----------------------------------------------------|----------|---------------|---------------------------|-----------------|-------------------|
| <input type="checkbox"/> activo         | Examinar Estructura Buscar Insertar Vaciar Eliminar | 0        | InnoDB        | utf8mb4_spanish_ci        | 32.0 KB         | -                 |
| <input type="checkbox"/> proyecto       | Examinar Estructura Buscar Insertar Vaciar Eliminar | 0        | InnoDB        | utf8mb4_spanish_ci        | 48.0 KB         | -                 |
| <input type="checkbox"/> responsable    | Examinar Estructura Buscar Insertar Vaciar Eliminar | 0        | InnoDB        | utf8mb4_spanish_ci        | 32.0 KB         | -                 |
| <input type="checkbox"/> seguimiento    | Examinar Estructura Buscar Insertar Vaciar Eliminar | 0        | InnoDB        | utf8mb4_spanish_ci        | 48.0 KB         | -                 |
| <input type="checkbox"/> usuario        | Examinar Estructura Buscar Insertar Vaciar Eliminar | 0        | InnoDB        | utf8mb4_spanish_ci        | 32.0 KB         | -                 |
| <input type="checkbox"/> vulnerabilidad | Examinar Estructura Buscar Insertar Vaciar Eliminar | 0        | InnoDB        | utf8mb4_spanish_ci        | 48.0 KB         | -                 |
| <b>6 tablas</b>                         | <b>Número de filas</b>                              | <b>0</b> | <b>InnoDB</b> | <b>utf8mb4_spanish_ci</b> | <b>240.0 KB</b> | <b>0 B</b>        |

Figura 13. Base vulnerabilidades\_db con sus seis tablas en phpMyAdmin. Captura propia.

### 13.5 Inserción de registros

El script 02\_insercion.sql carga datos de prueba ficticios (tres cuentas, tres proyectos, seis activos, cuatro responsables, seis hallazgos en distintos estados y seis seguimientos, uno con reapertura). Ningún dato corresponde a hallazgos reales de la organización, porque publicar vulnerabilidades reales en un repositorio abierto sería justamente la fuga de información que el sistema busca evitar.

✓ 6 filas insertadas.  
La id de la fila insertada es: 6 (La consulta tardó 0,0002 segundos.)

```
-- Uno cerrado, uno en tratamiento, uno reabierto, dos pendientes y uno sin evaluar. INSERT INTO vulnerabilidad (id_activo, id_responsable, titulo, descripcion, tipo, elemento_afectado, fecha_deteccion, fecha_objetivo, estado, vector_cvss) VALUES (1, 3, 'TLS 1.0 y 1.1 habilitados', 'El servidor acepta versiones de TLS obsoletas.', 'Configuración insegura', 'Puerto 443/TCP', '2026-08-18', '2026-09-15', 'CERRADA', 'CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:N/A:N'), (3, 2, 'Inyección SQL en la búsqueda de expedientes', 'El parámetro de búsqueda se concatena en la consulta.', 'Inyección', 'Parámetro q de /expedientes/buscar', '2026-09-02', '2026-09-30', 'EN_TRATAMIENTO', 'CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N'), (2, 2, 'XSS reflejado en el formulario de contacto', 'El campo
```

[ Editar ]

✓ Mostrando filas 0 - 5 (total de 6, La consulta tardó 0,0002 segundos.)

```
SELECT 'usuario' AS tabla, COUNT(*) AS filas FROM usuario UNION ALL SELECT 'proyecto', COUNT(*) FROM proyecto UNION ALL SELECT 'activo', COUNT(*) FROM activo UNION ALL SELECT 'responsable', COUNT(*) FROM responsable UNION ALL SELECT 'vulnerabilidad', COUNT(*) FROM vulnerabilidad UNION ALL SELECT 'seguimiento', COUNT(*) FROM seguimiento;
```

☐ Perfilando [ Editar en línea ] [ Editar ] [ Explicar SQL ] [ Crear código PHP ] [ Actualizar ]

☐ Mostrar todo | Número de filas: 25 | Filtrar filas: Buscar en esta tabla

Opciones extra

| tabla          | filas |
|----------------|-------|
| usuario        | 3     |
| proyecto       | 3     |
| activo         | 6     |
| responsable    | 4     |
| vulnerabilidad | 6     |
| seguimiento    | 6     |

Figura 14. Ejecución del script de inserción en phpMyAdmin y cantidad de filas por tabla. Captura propia.

### 13.6 Consulta de registros

La consulta principal del administrador (CU05) une vulnerabilidad, activo y proyecto, y usa LEFT JOIN con responsable porque un hallazgo puede no tener responsable todavía.

✓ Mostrando filas 0 - 4 (total de 5, La consulta tardó 0,0008 segundos.)

```
SELECT v.id_vulnerabilidad AS id, p.nombre AS proyecto, a.nombre AS activo, v.titulo, v.estado, r.nombre AS responsable, v.fecha_objetivo FROM vulnerabilidad v JOIN activo a ON a.id_activo = v.id_activo JOIN proyecto p ON p.id_proyecto = a.id_proyecto LEFT JOIN responsable r ON r.id_responsable = v.id_responsable WHERE v.estado <> 'CERRADA' ORDER BY v.fecha_deteccion;
```

☐ Perfilando [ Editar en línea ] [ Editar ] [ Explicar SQL ] [ Crear código PHP ] [ Actualizar ]

| id | proyecto                   | activo                       | titulo                                           | estado         | responsable        | fecha_objetivo |
|----|----------------------------|------------------------------|--------------------------------------------------|----------------|--------------------|----------------|
| 2  | Gestión de expedientes     | Aplicación de expedientes    | Inyección SQL en la búsqueda de expedientes      | EN_TRATAMIENTO | Desarrollo interno | 2026-09-30     |
| 3  | Portal web institucional   | Formulario de contacto       | XSS reflejado en el formulario de contacto       | PENDIENTE      | Desarrollo interno | 2026-10-05     |
| 4  | Infraestructura de oficina | NAS de respaldos             | Panel del NAS con credenciales por defecto       | PENDIENTE      | NULL               | NULL           |
| 5  | Portal web institucional   | Servidor web                 | Divulgación de versión en encabezados HTTP       | PENDIENTE      | NULL               | NULL           |
| 6  | Gestión de expedientes     | Base de datos de expedientes | Conexión a la base sin cifrado en la red interna | PENDIENTE      | NULL               | NULL           |

Figura 15. Consulta de hallazgos abiertos con JOIN en phpMyAdmin. Captura propia.

El script 03\_consultas.sql tiene además el listado del cliente (para el cliente 3 devuelve solo los hallazgos 2, 4 y 6, que son de sus dos proyectos), el historial del hallazgo 3 con su reapertura, la cantidad de hallazgos por proyecto y estado, y la asignación del hallazgo 4 con el inicio de su tratamiento dentro de una transacción (UPDATE del hallazgo e INSERT del seguimiento con un único COMMIT). Al final intenta poner una fecha objetivo anterior a la detección y la base lo rechaza con el error 4025 de la restricción ck\_fecha\_objetivo.

### 13.7 Borrado de registros

El script 04\_borrado.sql borra un responsable sin hallazgos, que se permite, e intenta borrar un activo que sí tiene. La clave foránea lo impide con el error 1451, que es lo que protege el historial (si se pudiera borrar, sus hallazgos quedarían apuntando a un activo que no existe). Por último borra un activo sin hallazgos y esa operación se completa.

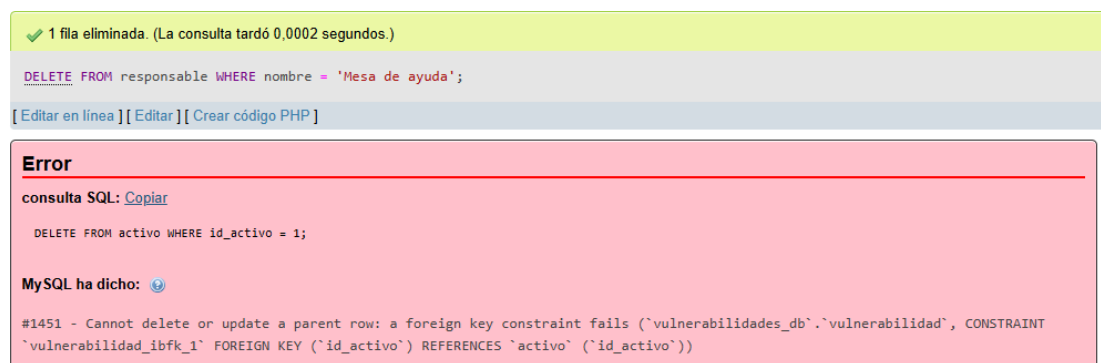


Figura 16. Borrado permitido y borrado rechazado por la clave foránea (error 1451) en phpMyAdmin. Captura propia.

### 13.8 Presentación de las consultas SQL

Siguiendo la indicación de no pegar todas las consultas en el informe, los scripts completos están en la carpeta TP2/sql del repositorio (<https://github.com/rodriguezd0/semprac-seguimiento-vulnerabilidades>), numerados en el orden de ejecución, y la salida real de cada uno está en TP2/salidas.

| Archivo          | Contenido                                                                                          |
|------------------|----------------------------------------------------------------------------------------------------|
| 01_creacion.sql  | Base con utf8mb4 y las seis tablas con sus claves y restricciones.                                 |
| 02_insercion.sql | Datos de prueba ficticios y cantidad de filas por tabla.                                           |
| 03_consultas.sql | Consultas de CU05, CU07 y CU13, resumen por estado, transacción de CU04 y CU06 y prueba del CHECK. |
| 04_borrado.sql   | Borrados permitidos y borrado rechazado por la clave foránea.                                      |

### 14. Definiciones de comunicación

El sistema no intercambia datos con sistemas externos, ya que toda la comunicación ocurre entre la aplicación Java y el servidor de base de datos. La aplicación usa JDBC con el protocolo cliente-servidor de MySQL sobre TCP, en el puerto 3306. En el prototipo los dos extremos están en el mismo equipo y en la operación real el tráfico viaja por la red interna del área.

| Capa            | Definición en el proyecto                                                                                                           |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------------|
| Aplicación      | JDBC con MySQL Connector/J y protocolo de MySQL, con sentencias preparadas.                                                         |
| Transporte      | TCP en el puerto 3306 del servidor (conexión orientada, con entrega ordenada y confirmada).                                         |
| Red             | IPv4 en la subred privada de la oficina, sin publicación hacia Internet.                                                            |
| Enlace y física | Ethernet (IEEE 802.3) con cable UTP categoría 5e o 6 hasta el switch del área, y Wi-Fi (IEEE 802.11) solo para equipos de consulta. |

El firewall del servidor filtra el puerto 3306 para que solo lo alcancen las PC del área. Como mejora queda habilitar TLS en la conexión JDBC para cifrar el tráfico dentro de la red (de hecho, uno de los hallazgos de prueba es una base que acepta conexiones sin cifrar).

### 15. Trazabilidad y referencias

La tabla sigue cada requerimiento a través de los artefactos del PUD, desde el caso de uso hasta la prueba y el prototipo.

| RF / CU           | Análisis                              | Diseño                                | Tablas                           | Prueba     | Prototipo |
|-------------------|---------------------------------------|---------------------------------------|----------------------------------|------------|-----------|
| RF03 / CU03       | Pantalla y gestor de registro         | Vulnerabilidad                        | vulnerabilidad                   | CP04       | Figura 11 |
| RF12 / CU12       | Calculadora CVSS                      | CalculadoraCvss31                     | vulnerabilidad                   | CP01, CP02 | Figura 11 |
| RF04 / CU04       | Gestor de cambio de estado            | Vulnerabilidad, asignar()             | vulnerabilidad                   | CP04       | -         |
| RF05 / CU05, CU13 | Pantalla y gestor de consulta         | Cliente, Proyecto                     | proyecto, activo, vulnerabilidad | CP05       | Figura 10 |
| RF06 / CU06       | Pantalla y gestor de cambio de estado | Vulnerabilidad, validarCambioEstado() | vulnerabilidad, seguimiento      | CP03       | -         |
| RF07 / CU07       | Gestor de consulta                    | Seguimiento                           | seguimiento                      | CP05       | -         |
| RF11 / CU11       | Pantalla y gestor de sesión           | Usuario, verificarClave()             | usuario                          | CP06       | Figura 9  |

El repositorio del proyecto es <https://github.com/rodriguezd0/semprac-seguimiento-vulnerabilidades> (carpeta TP2 con los scripts SQL, las salidas de ejecución, los diagramas, los prototipos y las pruebas).

### Referencias

FIRST. (2019a). *Common Vulnerability Scoring System v3.1: Specification document*.  
<https://www.first.org/cvss/v3.1/specification-document>

FIRST. (2019b). *Common Vulnerability Scoring System v3.1: Examples*.  
<https://www.first.org/cvss/v3.1/examples>

Kendall, K. E., y Kendall, J. E. (2011). *Análisis y diseño de sistemas* (8.ª ed.). Pearson Educación.

Ley 25.326 de Protección de los Datos Personales. (2000). Boletín Oficial de la República Argentina.

Universidad Siglo 21. (s. f.-a). *Sistemas de información* [Lectura 1, Seminario de Práctica de Informática]. Canvas, curso 46312.

Universidad Siglo 21. (s. f.-b). *Proceso unificado de desarrollo* [Lectura 2, Seminario de Práctica de Informática]. Canvas, curso 46312.

## Anexo A. Preguntas de elicitación

Las respuestas sintetizan el relevamiento que hice en el área tecnológica durante septiembre de 2026, sin nombres ni datos personales de los participantes.

### A.1 Entrevista al responsable del área tecnológica

| Pregunta                                               | Síntesis de la respuesta                                                                                                       |
|--------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| 1. ¿Cómo se detectan hoy los hallazgos de seguridad?   | Por revisiones de configuración, alertas de los equipos y avisos de usuarios.                                                  |
| 2. ¿Dónde se registran y qué datos se anotan?          | En una planilla del área, con el recurso, la descripción y una gravedad estimada. Responsable y avances quedan en los correos. |
| 3. ¿Quién decide la gravedad y con qué criterio?       | Quien anota el hallazgo, según su experiencia. No hay un método común.                                                         |
| 4. ¿Quién corrige y cómo se asigna la tarea?           | Depende del recurso (soporte interno, desarrollo o el proveedor de hosting) y se avisa por correo o mensajería.                |
| 5. ¿Cuándo se considera cerrado un hallazgo?           | Cuando se verifica la corrección sobre el activo, por ejemplo repitiendo la prueba que lo detectó.                             |
| 6. ¿Qué pasa si un hallazgo cerrado vuelve a aparecer? | Se anota como una fila nueva y se pierde la relación con el caso anterior.                                                     |
| 7. ¿Qué información no debería ver un referente?       | Los hallazgos de proyectos que no son suyos.                                                                                   |
| 8. ¿Qué método de severidad resultaría aceptable?      | Uno estándar y reconocido. Se acordó usar CVSS 3.1 base.                                                                       |

### A.2 Cuestionario a referentes de proyecto

| Pregunta                                                                           | Síntesis de las respuestas                                                                 |
|------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| 1. ¿Con qué frecuencia necesita conocer el estado de los hallazgos de su proyecto? | Una vez por semana o cuando hay un cambio importante.                                      |
| 2. ¿Qué datos necesita ver?                                                        | Título, estado, severidad, responsable y fecha objetivo. El detalle técnico, solo a veces. |
| 3. ¿Le sirve ver el historial de cambios?                                          | Sí, para saber qué se hizo y por qué se reabrió algo.                                      |
| 4. ¿Necesita modificar datos?                                                      | No, solo consultar.                                                                        |